

COMPUTER ORGANIZATION

Unit-I: Data Representation

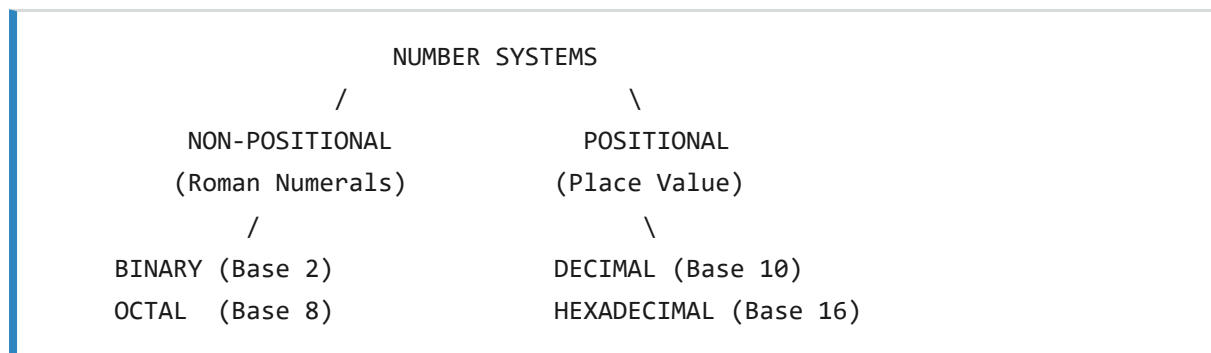
Comprehensive Study Notes

HPU BCA Semester III

1. Number Systems

1.1 What is a Number System?

A **number system** is a mathematical notation for representing numbers of a given set using digits or symbols. Computers use different number systems to represent data.



1.2 Common Number Systems

System	Base	Digits/Symbols	Example
Binary	2	0, 1	1010 ₂
Octal	8	0–7	25 ₈
Decimal	10	0–9	42 ₁₀
Hexadecimal	16	0–9, A–F	2A ₁₆

1.3 Decimal Number System (Base 10)

- Uses digits **0 to 9**
- Each position represents a power of 10

$$375_{10} = 3 \times 10^2 + 7 \times 10^1 + 5 \times 10^0$$

1.4 Binary Number System (Base 2)

- Uses digits **0 and 1**
- Each position represents a power of 2

$$1010_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 10_{10}$$

2. Number System Conversions

2.1 Decimal to Binary Conversion

Method: Successive division by 2 (remainders give binary digits from LSB to MSB)

Example: Convert 25_{10} to Binary

```
25 ÷ 2 = 12, Remainder = 1 (LSB)
12 ÷ 2 = 6, Remainder = 0
6 ÷ 2 = 3, Remainder = 0
3 ÷ 2 = 1, Remainder = 1
1 ÷ 2 = 0, Remainder = 1 (MSB)
```

Reading from bottom to top: $25_{10} = 11001_2$

2.2 Binary to Decimal Conversion

Method: Multiply each digit by its positional weight (power of 2)

Example: Convert 11001_2 to Decimal

$$11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 0 + 0 + 1 = 25_{10}$$

2.3 Decimal to Octal Conversion

Method: Successive division by 8

Example: Convert 100_{10} to Octal

$100 \div 8 = 12$, Remainder = 4 (LSB)
 $12 \div 8 = 1$, Remainder = 4
 $1 \div 8 = 0$, Remainder = 1 (MSB)

Reading from bottom to top: $100_{10} = 144_8$

2.4 Octal to Decimal Conversion

Example: Convert 144_8 to Decimal

$$144_8 = 1 \times 8^2 + 4 \times 8^1 + 4 \times 8^0 = 64 + 32 + 4 = 100_{10}$$

2.5 Decimal to Hexadecimal Conversion

Method: Successive division by 16

Example: Convert 100_{10} to Hexadecimal

$100 \div 16 = 6$, Remainder = 4 (LSB)
 $6 \div 16 = 0$, Remainder = 6 (MSB)

Reading from bottom to top: $100_{10} = 64_{16}$

2.6 Hexadecimal to Decimal Conversion

Example: Convert $2A_{16}$ to Decimal

$$2A_{16} = 2 \times 16^1 + 10 \times 16^0 = 32 + 10 = 42_{10}$$

2.7 Binary to Octal Conversion

Method: Group binary digits in groups of 3 from LSB

Example: Convert 101110_2 to Octal

Binary: 101 110
Octal: 5 6
 $101110_2 = 56_8$

2.8 Binary to Hexadecimal Conversion

Method: Group binary digits in groups of 4 from LSB

Example: Convert 101110_2 to Hexadecimal

Binary: 0010 1110
Hex: 2 E
 $101110_2 = 2E_{16}$

2.9 Conversion Summary Table

From	To	Method
Decimal	Binary	Divide by 2
Decimal	Octal	Divide by 8
Decimal	Hex	Divide by 16
Binary	Octal	Group by 3
Binary	Hex	Group by 4
Octal	Decimal	Multiply by powers of 8
Hex	Decimal	Multiply by powers of 16

2.10 Solved Examples

Example 1: Convert 45_{10} to Binary

$45 \div 2 = 22, R = 1$
 $22 \div 2 = 11, R = 0$
 $11 \div 2 = 5, R = 1$
 $5 \div 2 = 2, R = 1$
 $2 \div 2 = 1, R = 0$
 $1 \div 2 = 0, R = 1$

$45_{10} = 101101_2$

Example 2: Convert 101101_2 to Decimal

$$101101_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 32 + 0 + 8 + 4 + 0 + 1 = 45_{10}$$

Example 3: Convert 11010110_2 to Octal

Binary: 011 010 110
Octal: 3 2 6
 $11010110_2 = 326_8$

Example 4: Convert 11010110_2 to Hexadecimal

Binary: 1101 0110
Hex: D 6
 $11010110_2 = D6_{16}$

3. Binary Coded Decimal (BCD)

3.1 What is BCD?

BCD (Binary Coded Decimal) represents each decimal digit using a 4-bit binary code.

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Decimal 349 = BCD: 0011 0100 1001

Note: BCD uses only 0000 to 1001 (0–9). 1010 to 1111 are invalid in BCD.

3.2 BCD Addition

Rules:

1. Add the two BCD numbers as binary numbers
2. If $\text{sum} \leq 9$, it is valid
3. If $\text{sum} > 9$, add 6 (0110) to correct

Example: Add 3 and 9 in BCD

```
  0011 (3)
+ 1001 (9)
-----
```



```
1100 (> 9, so add 6)
+ 0110
-----
0001 0010 (12 in BCD = 0001 0010)
```

4. Hamming Code

4.1 What is Hamming Code?

Hamming code is an error-detecting and error-correcting code used in digital communication and memory systems.

- Can detect and correct single-bit errors
- Uses parity bits at specific positions
- Positions are powers of 2 (1, 2, 4, 8, ...)

4.2 Hamming Code Structure

Position	1	2	3	4	5	6	7	8	9	10	11
Type	P1	P2	D1	P4	D2	D3	D4	P8	D5	D6	D7

Where: P = Parity bit, D = Data bit

4.3 Parity Bit Calculation

Parity Bit	Covers Positions
P1	1, 3, 5, 7, 9, 11
P2	2, 3, 6, 7, 10, 11
P4	4, 5, 6, 7
P8	8, 9, 10, 11

Calculation: Each parity bit is set to make the total number of 1s in the covered positions even (even parity).

4.4 Solved Example: Encode Data 1010

Step 1: Positions: D1=1, D2=0, D3=1, D4=0

Positions: 1(P1), 2(P2), 3(1), 4(P4), 5(0), 6(1), 7(0)

Step 2: Calculate parity bits

P1: Covers 1,3,5,7 → 1, 1, 0, 0 → P1 = 0 (even parity)

P2: Covers 2,3,6,7 → 1, 1, 1, 0 → P2 = 1 (even parity)

P4: Covers 4,5,6,7 → 0, 0, 1, 0 → P4 = 1 (even parity)

Step 3: Final code = 0 1 1 1 0 1 0 = 0111010

4.5 Error Detection

- If a bit error occurs, recalculate parity bits
- The parity bits form a binary number indicating the error position

Example: If the error position is 5 (101 in binary), then P4=1, P2=0, P1=1

5. Character Encoding Schemes

5.1 ASCII (American Standard Code for Information Interchange)

ASCII is a 7-bit or 8-bit code used to represent characters.

ASCII Code	Character	ASCII Code	Character
65	A	97	a
66	B	98	b
67	C	99	c

Extended ASCII: Uses 8 bits (0–255) for additional characters.

5.2 EBCDIC

Extended Binary Coded Decimal Interchange Code – 8-bit character encoding used primarily in IBM mainframe computers.

5.3 Unicode

- Universal character encoding standard
- Supports multiple languages
- Uses 16-bit or 32-bit encoding
- Covers over 128,000 characters

6. Binary Arithmetic

6.1 Binary Addition

Rules:

```
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 (carry 1)
1 + 1 + 1 = 1 (carry 1)
```

Example: $1011_2 + 0110_2$

```
  1011
+ 0110
-----
 10001
```

6.2 Binary Subtraction

Rules:

```
0 - 0 = 0
1 - 1 = 0
1 - 0 = 1
0 - 1 = 1 (borrow 1)
```

Example: $1010_2 - 0110_2$

```
  1010
- 0110
-----
  0100
```

6.3 Binary Multiplication

Rules: Same as decimal multiplication – multiply by each digit and add partial products.

Example: $1011_2 \times 101_2$

```

      1011
x     101
-----
      1011
     0000
+    1011
-----
    110111

```

6.4 Binary Division

Example: $10101_2 \div 11_2$

```

      111
    -----
11) 10101
    11
    ---
    100
    11
    ---
    11
    11
    --
    0

Answer:  $111_2$ 

```

7. Complements

7.1 1's Complement

Invert all bits ($0 \rightarrow 1$, $1 \rightarrow 0$).

1's complement of 1010 is 0101

7.2 2's Complement

2's complement = 1's complement + 1

2's complement of 1010: $0101 + 1 = 0110$

7.3 Addition/Subtraction using 2's Complement

Steps for Subtraction using 2's Complement:

1. Take 2's complement of the subtrahend
2. Add to the minuend
3. Ignore carry (if any)

Example: 1101 - 1011

```
Minuend:           1101
Subtrahend:        1011
2's complement of 1011: 0101
Add: 1101 + 0101 = 10010
Ignore carry (1):   0010

Answer: 00102
```

7.4 9's and 10's Complement (Decimal)

- **9's complement:** Subtract each digit from 9
- **10's complement:** 9's complement + 1

9's complement of 375 = $999 - 375 = 624$

10's complement of 375 = $624 + 1 = 625$

8. Floating Point Representation

8.1 Floating Point Concept

$$\text{Number} = \pm \text{Mantissa} \times \text{Base}^{\text{Exponent}}$$

Example: $32.5 = 3.25 \times 10^1$

8.2 IEEE 754 Standard

Format	Bits	Sign	Exponent	Mantissa
Single Precision	32	1	8	23
Double Precision	64	1	11	52

8.3 Floating Point Addition and Subtraction

Steps:

1. Align exponents (make them equal)
2. Add or subtract mantissas
3. Normalize the result
4. Round if necessary

Example: Add 32.5 and 2.5

$$\begin{aligned} 32.5 &= 3.25 \times 10^1 \\ 2.5 &= 2.50 \times 10^0 \end{aligned}$$

$$\text{Align: } 2.5 \times 10^0 = 0.25 \times 10^1$$

$$\text{Add: } 3.25 \times 10^1 + 0.25 \times 10^1 = 3.5 \times 10^1 = 35.0$$

9. Signed Binary Numbers

9.1 Sign-Magnitude Representation

- MSB represents sign (0 = positive, 1 = negative)
- Remaining bits represent magnitude

+5 = 0101, -5 = 1101 (4-bit system)

9.2 1's Complement Representation

- Positive: same as sign-magnitude
- Negative: invert all bits of the positive number

+5 = 0101, -5 = 1010 (1's complement)

9.3 2's Complement Representation

- Negative: 2's complement of the positive number
- Most common representation in modern computers

+5 = 0101, -5 = 1011 (2's complement)

9.4 Range of Signed Numbers (n-bit)

Representation	Range
Sign-Magnitude	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$
1's Complement	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$

2's Complement	-2^{n-1} to $+(2^{n-1} - 1)$
----------------	--------------------------------

Questions

Short Answer Questions (2–5 Marks)

1. What are the different number systems?
2. Convert 45_{10} to binary.
3. Convert 1101_2 to decimal.
4. What is BCD?
5. What is Hamming code?
6. What is 2's complement?
7. What is ASCII?
8. What is binary addition?
9. What is floating point representation?
10. What are signed binary numbers?

Long Answer Questions (10 Marks)

1. Explain different number systems and conversion methods with examples.
2. Explain BCD addition with examples.
3. Explain Hamming code for error detection and correction.
4. Explain 1's and 2's complement addition and subtraction with examples.
5. Explain floating point addition and subtraction with example.
6. Explain signed binary number representations (sign-magnitude, 1's complement, 2's complement).

Objective Questions (1 Mark)

1. Binary uses base:
(a) 2 (b) 8 (c) 10 (d) 16
2. The BCD code for 9 is:
(a) 1001 (b) 1010 (c) 1000 (d) 1111
3. ASCII stands for:
(a) American Standard Code for Information Interchange

- (b) American Standard Code for International Interchange
 - (c) Both (d) None
4. 2's complement of 1010 is:
- (a) 0101 (b) 0110 (c) 1011 (d) 1100
5. Which number system uses base 16?
- (a) Binary (b) Octal (c) Decimal (d) Hexadecimal

Quick Revision

Number System Conversion Summary

From	To	Method
Decimal	Binary	Divide by 2
Binary	Decimal	Multiply by powers of 2
Binary	Octal	Group by 3
Binary	Hex	Group by 4

BCD Table

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Key Formulas

1's Complement: Invert all bits

2's Complement: 1's Complement + 1

$$\text{Floating Point: } \pm \text{ Mantissa} \times \text{Base}^{\text{Exponent}}$$

End of Unit-I Notes